

Connect to the conference wi-fi:

Network: dbt Summit

Passcode: run_dbt!

**Once you're seated, please join the WiFi – this
helps everyone connect smoothly!**

**Access AI closed captions and
translation on your personal device.**



Getting Started with dbt

We're excited to walk through the fundamentals of dbt with you today hands-on, in dbt Platform



Shania Thomas

she/her

Sr. Technical Instructor
dbt Labs

Middle school science > data analysis > technical instruction



David Elliott

he/him

Sr. Tech Instructor
dbt Labs

SE -> data science instructor -> data engineer -> now!

Meet your helpful dbt Labs TAs!



Faith McKenna

she/her

Manager, Technical Instruction



Lucas Silbernagel

he/him

Resident Architect



And you!

Be a great neighbor - share,
connect, help each other

ICEBREAKER

Introduce yourself to your tablemates

- Name
- Pronouns (if comfortable)
- Role
- Location
- Your dbt, SQL & Git experience (1 = totally new, 3 = extensive experience)

NORMS

We'll ask you to be... and you're encouraged to...

We'll ask you to be

 All in- stay engaged, participate

 Community-minded - dive in, engage in dbt Community Slack

 Curious - ask questions, it's ok to be wrong

You're encouraged to

 Take care of your needs - water, snacks, stretch breaks

 Ask a question - if you're thinking it, others probably are too

 Flag a TA if you fall behind - we'll circulate

TODAY'S AGENDA

Agenda

30 min Introduction to dbt

15 min Setting up dbt

15 min *Break*

20 min Sources

30 min Data Modeling in dbt

15 min *Break*

30 min Testing

15 min Documentation

10 min *Break*

25 min Deployment and dbt Catalog

15 min Wrap Up

HOW WE'LL LEARN

Learning Methods

Content Based

- **Instructor-led:** introduction of new concepts
- **Demonstration in dbt**
- **Checkpoints:** pausing for questions

Practice Based

- **Learner-led:** knowledge checks
- **Collaborative:** share the floor - step up, step back

COURSE OBJECTIVES

By the end of this course you'll be able to...

01

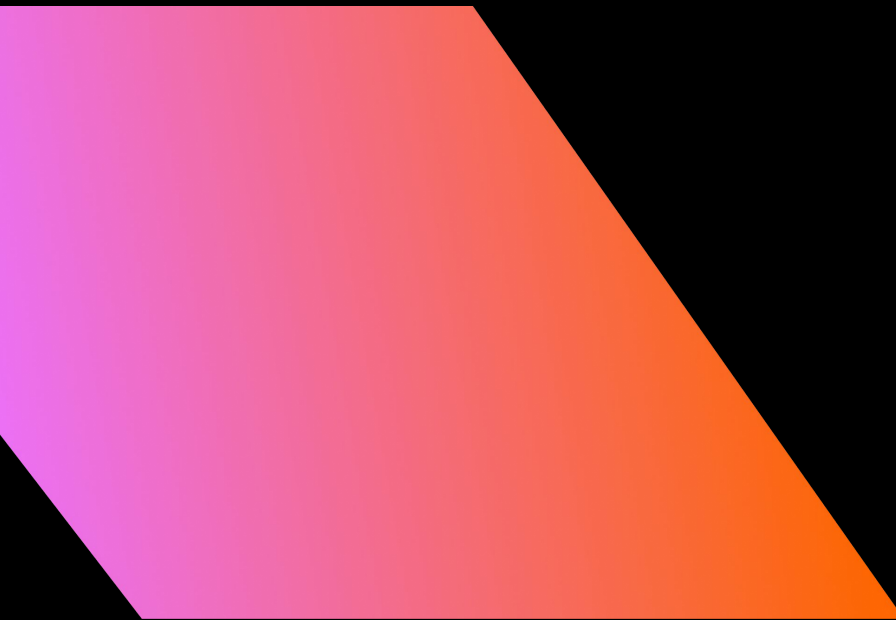
Explain *why* dbt

02

Create sources,
write tests, and
develop
documentation

03

Navigate dbt
Platform's features
to complete a full
data workflow



Questions?

TODAY'S AGENDA

Agenda

30 min Introduction to dbt

15 min Setting up dbt

15 min *Break*

20 min Sources

30 min Data Modeling in dbt

15 min *Break*

30 min Testing

15 min Documentation

10 min *Break*

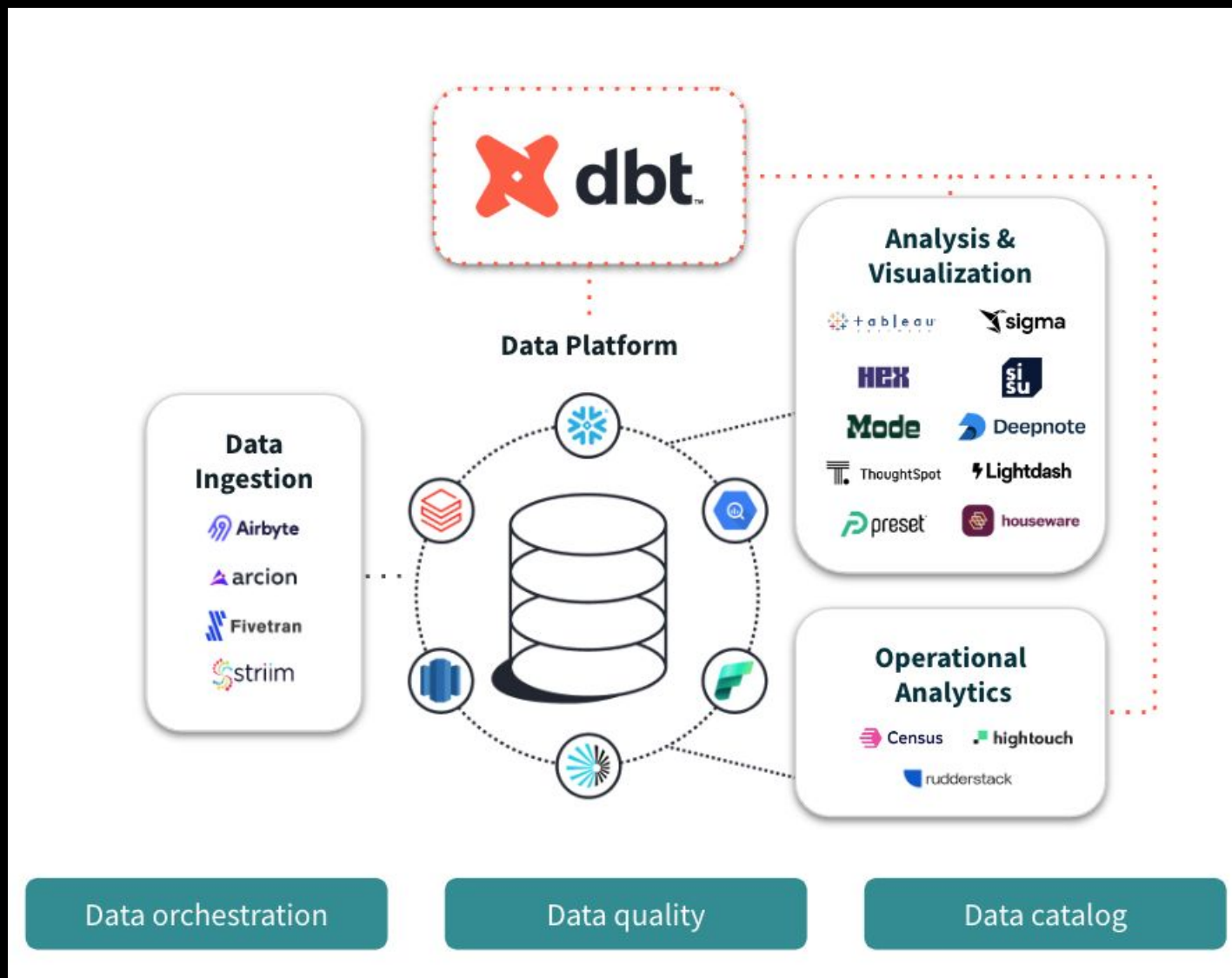
25 min Deployment and dbt Catalog

15 min Wrap Up



Why dbt?

THE MODERN DATA STACK



THE COST OF SKIPPING THIS

Modern data stack: consequences



Ambiguous lineage



Poor documentation



Time-consuming, difficult updates



Delicate queries



Lack of testing



No collaboration

ENTER dbt

ELT & dbt



Best Practices

Software Engineering

dbt applies software engineering best practices to analytics: version control, logging, alerting, automated testing and deployment.



SQL & Canvas

Accessibility

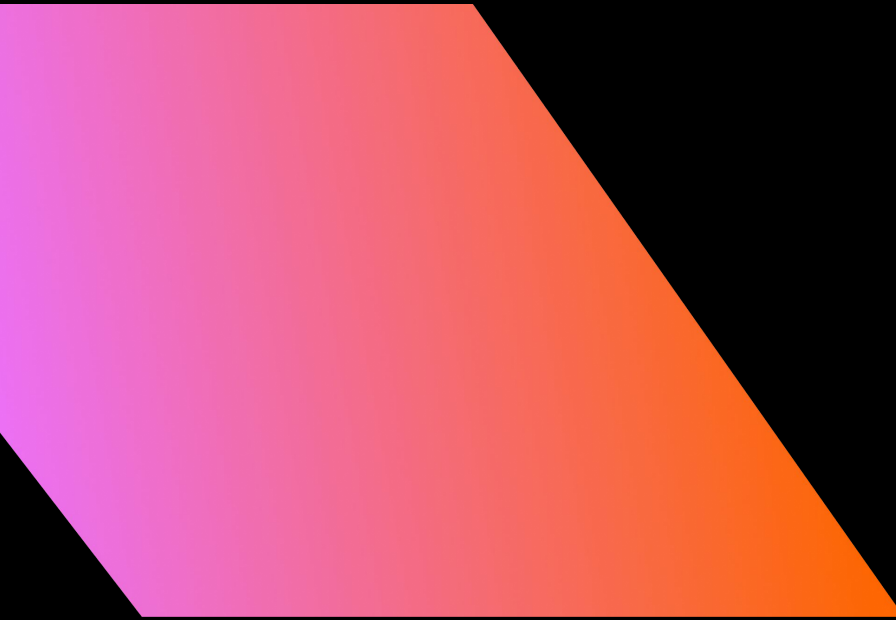
Anyone who knows SQL can do this — and with Canvas/Insights, so can those who don't.



A New Discipline

Analytics Engineering

This role is neither data engineering nor data analysis — it's **analytics engineering**.



Questions?

SETTING THE SCENE

Jaffle Shop



Sandbox Env

Sample Data

We're in a sandbox using sample data from the fictional Jaffle Shop.



The Role

Data Analyst

I'm acting as an analyst at the Jaffle Shop to explore operational data.



Order App

Management

We use our own custom application for database and order management.



What is it?

Jaffle Definition

What exactly is a **"Jaffle"**? Does anyone have any guesses?



Scenario

My goal: total orders per customer



Available Data

Customers, Orders



BI Usability

Can this data be used as-is to create a BI visual?



AI Capabilities

Get a better answer from AI?



Next Steps

If not - what does it need?



Transform to...

01

Create clarity

Establish clear business definitions for critical model columns (e.g. what is `id`?).

02

Build fields

Calculate new fields and metrics (e.g. `number_of_orders` aggregated by customer).

03

Create table

Materialize a new table named `dim_customers` to store your final transformed data.

04

Connect tools

Seamlessly expose and connect your BI or downstream tools directly to the newly transformed table.

THE MANUAL WAY

How to do this manually

What if the schema doesn't exist yet?

What if the schema exists and I need to recreate it?

What if the table already exists?

What if this depends on another transformation?

What if I want to parameterize the schema?

```
query.sql

create table
dbtlabs_jdoe.dim_customers as (

...

)
```



Using dbt

TO USE dbt YOU NEED

A supported data platform

Data Platform Flexibility

dbt acts as the transformation layer on top of your existing data infrastructure. It compiles and runs your code directly inside your target platform.

We are using **Snowflake** for this walkthrough, but the core concepts remain identical across all engines.



TO USE dbt YOU NEED

A connection to your data platform

Credentials

Contains the necessary credentials to safely authenticate and connect directly to your target data platform.

Schema & DB

Specifies the default target schema and database where dbt will compile and build your relational models.

TO USE dbt YOU NEED

A project



Git Repository

Connected directly to a Git repository to enable version control and collaboration



Initialization

Initialize to automatically build out the foundational dbt directory structure and sample **models**.



Project Config

Contains a **dbt_project.yml** file that serves as the entry point and master configuration for your project.

CONFIG FILES

What is .yaml?



Readable

A human friendly format designed to easily write and structure dictionaries and arrays.



Config Files

Used extensively to define configurations, documentation, and test parameters.



dbt Standard

Standardized throughout almost every layer of dbt projects.



Whitespace

Indentation is strict and can be tricky; copy and paste is highly encouraged!

MODELS

What are models?



Transform

SQL select statements written to transform and structure data.



Directory

Stored in the `models` directory, with a `.sql` extension.



Warehouse Relation

Maintains a direct 1:1 relationship with a data artifact in your warehouse.

TO USE dbt YOU NEED

dbt Commands

An instruction, issued from dbt Studio, to execute specific tasks in your project.



dbt run

Executes the SQL models to build relations and load tables in the data warehouse.



dbt build

Runs models, tests, snapshots, and seeds in directed acyclic graph (DAG) order.



dbt test

Executes tests defined on your schema and models to validate data quality.

UNDER THE HOOD

What happens when you run `dbt run`?



1. Parse

dbt parses your entire project to resolve dependencies and build the graph.



2. Connect

dbt connects to your specified data platform to run queries.



3. Wrap

dbt wraps your models in appropriate DDL/DML.



4. Execute

dbt executes this generated code to build your models in the target schema.

UNDER THE HOOD

What happens when you run `dbt run`?



1. Parse

dbt parses your entire project, ensuring jinja and yaml compiles correctly



2. Connect

dbt connects to your specified data platform to run queries.



3. Wrap

dbt wraps your models in appropriate DDL/DML like `create table as`.



4. Execute

dbt executes this generated code to build your models in the target schema.

CHECK FOR UNDERSTANDING

What are the some of the pieces you need to use dbt?



01

A data platform

The underlying database or cloud warehouse where your data resides and queries run.



02

dbt project

Directory containing your SQL models, YAML configuration files, and tests.



03

Warehouse connection

Profile settings and credentials used to connect dbt to your data platform.



04

Command

CLI operations like ``dbt run`` or ``dbt test`` to execute your transformations.

Questions?

TODAY'S AGENDA

Agenda

- 30 min Introduction to dbt
- 15 min** **Setting up dbt**
- 15 min *Break*
- 20 min Sources
- 30 min Data Modeling in dbt
- 15 min *Break*
- 30 min Testing
- 15 min Documentation
- 10 min *Break*
- 25 min Deployment and dbt Catalog
- 15 min Wrap Up

SET UP (15 MIN 🙌)

Setting up your environment

- Step 1: Installation
- Step 2: Clone Git repository
- Step 3: ``dbt init`` and ``profiles.yml``
- Step 4: Ensure connections
- Step 5: Create a working branch
- Step 6: Celebrate 🎉

REGISTRATION

Workshop Registration for dbt Platform

- 01 Log out of any dbt accounts, or use a private/incognito session
- 02 Go to tinyurl.com/dbt-workshop-setup
- 03 Select **"Getting Started with dbt"**
- 04 Enter the passcode **Summit2026!**
- 05 Take note of your User ID and Password – **screenshot it!**

STEP 1

Installation: VS Code & Fusion

- Download & install VS Code:
code.visualstudio.com/download
- Open VS Code
- Install Fusion in the terminal
- Run `dbtf --version`

```
# macOS/Linux
curl -fsSL
https://public.cdn.getdbt.com/fs/install/install.sh
| sh -s -- --update
exec $SHELL

# Windows/PowerShell (as Admin)
irm
https://public.cdn.getdbt.com/fs/install/install.ps1
| iex
Start-Process powershell
```

STEP 1

Installation: dbt VS Code Extension

- Open VS Code
- Go to the Extensions panel in the left sidebar
- Search for "dbt" in the search box
- Find the official "dbt" extension and click Install

STEP 2

Clone Git Repository

- Navigate to GitHub.com
- Search for the training repository
- Click <> Code and copy the HTTPS link

```
# In your terminal:  
git clone <HTTPS> dbt_training
```

STEP 3

`dbtf init` and `profiles.yml`

- Run `dbtf init` in the terminal
- Follow the prompts
- Schema = your personal dev schema: `dbt\_{first initial, last name}`

```
type: snowflake
account: yda41575
user: training_user
database: dbt_learn
warehouse: transformer
schema: <dbt_flast>
role: transformer
threads: 16
client_session_keep_alive: False
```

STEP 4

Ensure connections

- Run the following commands in the terminal:

```
dbtf compile  
dbtf run  
# or  
dbtf build
```

STEP 5

Create a working branch

- Open the command palette → "Git: Create Branch"
- Name your branch (e.g. `first\_last\_training`)
- Optional: `git push -u origin <branch-name>`
- Commit changes to git (e.g. "initial commit")

demo_time

Live in dbt Platform

KNOWLEDGE CHECK

Try it yourself

05:00

Hands-on Challenge

- **Create** a branch for this workshop
- **Find** `customers.sql` in your project
- **Build** using the command that materializes your models
- **View** details of the command execution

You should be able to

- Create a working development branch
- Locate specific models within the file explorer
- Execute standard dbt run and build commands

break_time

15 minutes

MATERIALIZATIONS

Materializations are build strategies

SQL Wrapper

The compiler wraps your SELECT statement in the appropriate target DDL/DML.

Configurable

You can easily configure and control a model's materialization strategy.

Multiple Strategies

Models can be materialized as **views, tables, incremental**, and more.

No Manual DDL

Without dbt, this would mean writing a lot of tedious DDL by hand.

CONFIGURATIONS

Configurations are model settings

Configuration Options

Flexible Definition: Set configurations globally in `dbt_project.yml` or locally within `model.yml` / `model.sql`.

Hierarchical Overrides: Settings are applied cascade-style. More specific, granular configurations always win over high-level defaults.

```
--dbt_project.yml
models:
  jaffle_shop:
    +materialized: table

--model.sql
{{
    config(
        materialized='table'
    )
}}
```

demo_time

Live in dbt Platform

KNOWLEDGE CHECK

Try it yourself

Hands-on

- Rename project
- Materialize in one file:
 - all models in **marts** as **table**
 - all models in **intermediate** as **table**
 - all models in **staging** as **view**
- **Build:** run the command that builds your models
- **Check:** verify all models materialized correctly

You should be able to

- Configure materializations in ``dbt_project.yml``
- Use the ``dbt run`` command

Questions?

TODAY'S AGENDA

Agenda

30 min Introduction to dbt

15 min Setting up dbt

15 min *Break*

20 min Sources

30 min Data Modeling in dbt

15 min *Break*

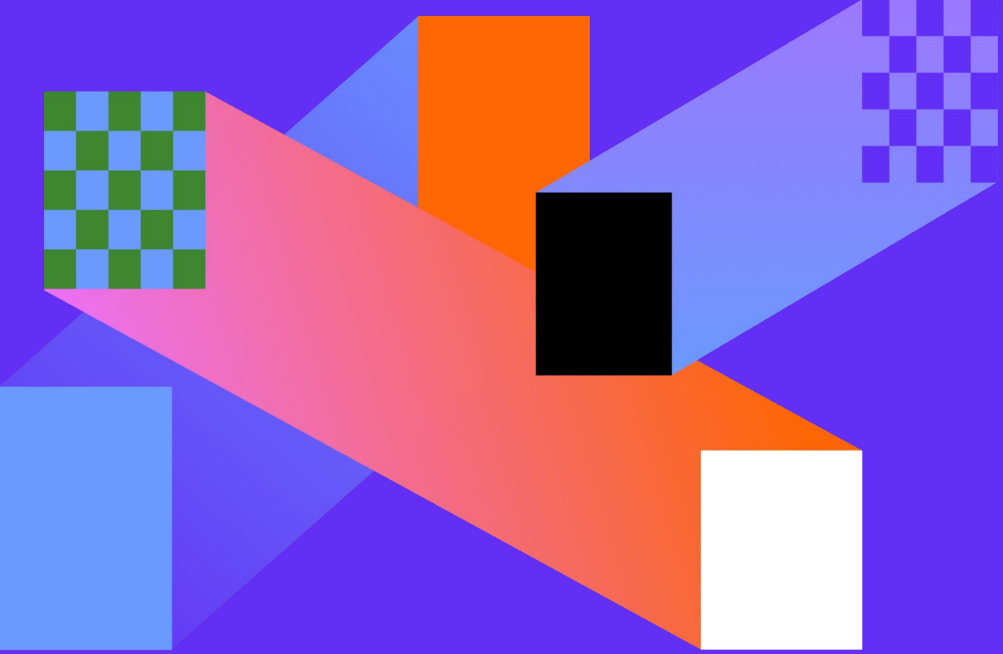
30 min Testing

15 min Documentation

10 min *Break*

25 min Deployment and dbt Catalog

15 min Wrap Up



Sources

WHY DECLARE SOURCES?

If you've ever had this happen...

 **Switching email marketing platforms** Which models depend on that source data?

 **Switching Redshift → Snowflake** All source data now lives in a different database/schema

 **Someone asks "where does this data really come from?"** Lack of clear data lineage and source mapping

 **Data loading hiccups** Dealing with delayed, incomplete, or duplicated data

 **A once-true assumption is no longer true** E.g., a new payment method or category breaks logic

BENEFITS

Sources may be right for you!



Declare & Update

Easily define and maintain the physical location of your raw data sources.

- Database & schema abstraction
- Dynamic name mapping



Trace Lineage

Establish a clear path from your raw database tables to downstream models.

- Auto-generated dependency graphs
- Visual source-to-model tracking



Validate Freshness

Keep tabs on your data pipeline health by verifying data arrival times.

- Customizable thresholds
- Proactive warning & error alerts

DEFINING SOURCES

Sources live in .yaml files in models/

name: How you refer to the source in dbt.

schema: The schema name in the database.
Defaults to name if not provided. *(Optional)*

database: Where the raw data lives in your warehouse.

tables: List of tables to declare as sources in this project.

name: How you'll refer to the source table in dbt.

identifier: Table name in your actual database, if different from name. *(Optional)*

```
_sources.yaml

sources:
- name: jaffle_shop
  database: raw
  schema: jaffle_shop_data
  tables:
    - name: customers
    - identifier: j_shop_cx
```

SELECTING FROM SOURCES

Use the `{{ source() }}` macro

```
model.sql

-- model.sql
with source as (
  select * from
  {{ source('jaffle_shop',
    'customers') }}
),
renamed as (...)
```



```
compiled_model.sql

-- compiled SQL
with source as (
  select * from
  raw.jaffle_shop.j_shop_cx
),
renamed as (...)
```

CHECK FOR UNDERSTANDING

What will this compile to?

```
select * from {{ source('event_data', 'events') }}
```

_sources.yml

sources:

- name: event_data
 database: raw
 schema: snowplow
 tables:
 - name: events

→

```
select * from raw.snowplow.events
```

CODEGEN

Setting up sources with codegen

01 Installation

Install the codegen package (hub.getdbt.com) to get access to source generation macros.

02 Configuration

Create a `packages.yml` file, paste the codegen config, and run `dbt deps`.

03 Generation

Use the `generate_source` macro to automatically compile the required schema code.

04 File Creation

Create the source YAML file `_src_jaffle_shop.yml` inside `models/staging`.

05 Save Results

Paste the generated configuration directly into your new file and save it to complete setup.

Using the generate_source macro

- Create `models/staging/\_src\_jaffle\_shop.yml`
- Run the command, redirecting output to that file
- Delete stray single quotes and save

```
dbt --quiet run-operation generate_source \  
  --args '{"schema_name": "jaffle_shop",  
         "database_name": "raw",  
         "table_names": ["orders","customers"],  
         "generate_columns": "true",  
         "include_descriptions": "true"}' \  
> models/staging/_src_jaffle_shop.yml
```

demo_time

Live in dbt Platform

KNOWLEDGE CHECK

Try it yourself

Hands-on Exercises

- Create `_sources.yml` in the staging folder
-

You should be able to

- Define sources with proper configuration
- Create modular staging models efficiently

Questions?

TODAY'S AGENDA

Agenda

30 min Introduction to dbt

15 min Setting up dbt

15 min *Break*

20 min Sources

30 min Data Modeling in dbt

15 min *Break*

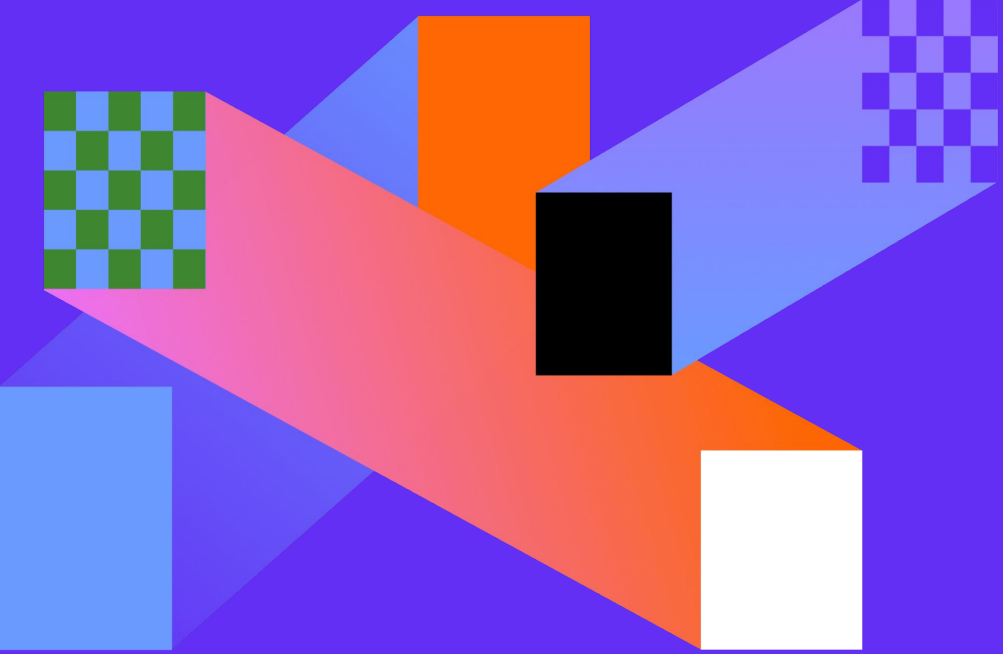
30 min Testing

15 min Documentation

10 min *Break*

25 min Deployment and dbt Catalog

15 min Wrap Up

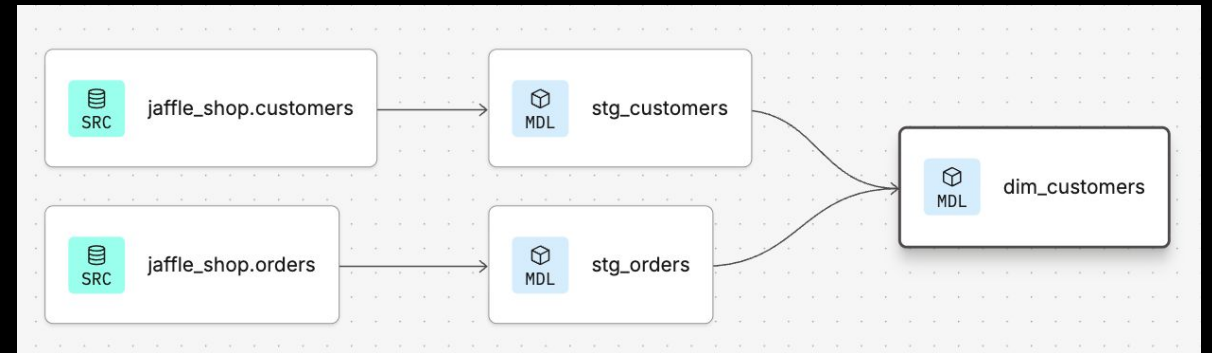


Data Modeling with dbt

PROJECT LINEAGE

Directed Acyclic Graphs (DAGs)

- **Maps complex pipelines:** Visualizes your entire data flow from source to end-model.
- **Impact analysis:** Instantly trace upstream sources and downstream dependencies.
- **Easier debugging:** Pinpoint pipeline failures and audit data lineage step-by-step.
- **Macro-driven:** Automatically constructed using native dbt relationships.



Key Lineage Macros

source() Declares and tracks raw data entry points

ref() Automatically builds modular dependencies

DEPENDENCIES

Creating dependencies with ref()

Dynamic Compilation

Safely references other models. During execution, dbt dynamically resolves the correct target schema and table name for your active environment.

Automated Run Order

By referencing models using ref(), dbt automatically constructs a Directed Acyclic Graph (DAG) to determine optimal parallel execution sequence.

```
-- model.sql
```

```
select * from {{  
  ref('stg_orders')  
}}
```



```
--compiled SQL
```

```
select * from  
analytics_dev.stg_o  
rders
```

NAMING CONVENTIONS

The dbt DAG and model prefixes

stg_

- 1:1 with raw data tables.
- Light transformations: renaming, type casting, basic computations, date transforms.

int_

- Allows DRY
- Used by any model downstream
- Break complex CTE up

NAMING CONVENTIONS

The dbt DAG and model prefixes

dim_

- "Nouns" - core business entities
- customers, products, employees.
- Some slowly-changing attributes (address, email).

fct_

- "Verbs" - core business processes: orders, cases, plays.
- Often built on an immutable event stream.

ORGANIZATION

Folder Structure

- Files: follow a naming convention
- Within files: adhere to a SQL style guide
- Between files: better organization = easier onboarding + collaboration

```
|— README.md
|— dbt_project.yml
└─ models
    |— marts
    |   |— _core.yml
    |   |— dim_customers.sql
    |   └─ fct_orders.sql
    └─ staging
        |— jaffle_shop
        |   |— _src_jaffle_shop.yml
        |   |— stg_jaffle_shop__customers.sql
        |   └─ stg_jaffle_shop__orders.sql
        └─ stripe
            |— _src_stripe.yml
            └─ stg_stripe_payments.sql
```

CODEGEN

Creating Staging Models from Sources

OPTION 1

'generate model' button

An automated, click-to-generate option directly within the dbt Cloud IDE interface to instantly scaffold staging models from your sources.

OPTION 2

Use codegen package 'generate_base_model'

1. Ensure the package is declared in `packages.yml`
2. Copy the generation macro code from `hub.getdbt.com` into a new file
3. Replace the placeholder source and table names with your own
4. Run the macro, then save the compiled output SQL in `models/staging`

demo_time

Live in dbt Platform

KNOWLEDGE CHECK

Try it yourself

Hands-on Exercises

- **Create:** Create staging models for orders and customers.
- **Rename:** Do appropriate renames to columns (i.e. id > order_id)
- **Replace:** Update CTEs in `dim_customers.sql` with references using the `ref()` function to staging models and correct names

You should be able to

- Define sources with proper configuration
- Create modular staging models efficiently

FILE EXPLORER

dim_customers

File Structure

Your file tree should look like this:

File explorer

- dbt-summit-26-Getting-Started-with-dbt
 - logs
 - macros
 - models
 - mart
- dim_customers.sql

SQL Model Code

Your dim_customers model should look like this:

```
with customers as (  
  
  select  
    customer_id,  
    first_name,  
    last_name  
  
  from {{ ref('stg_jaffle_shop__customers') }}  
  
)  
  
Preview CTE  
orders as (  
  
  select  
    order_id,  
    customer_id,  
    order_date,  
    status  
  
  from {{ ref('stg_jaffle_shop__orders') }}
```

Questions?

break_time

15 minutes

TODAY'S AGENDA

Agenda

30 min Introduction to dbt

15 min Setting up dbt

15 min *Break*

20 min Sources

30 min Data Modeling in dbt

15 min *Break*

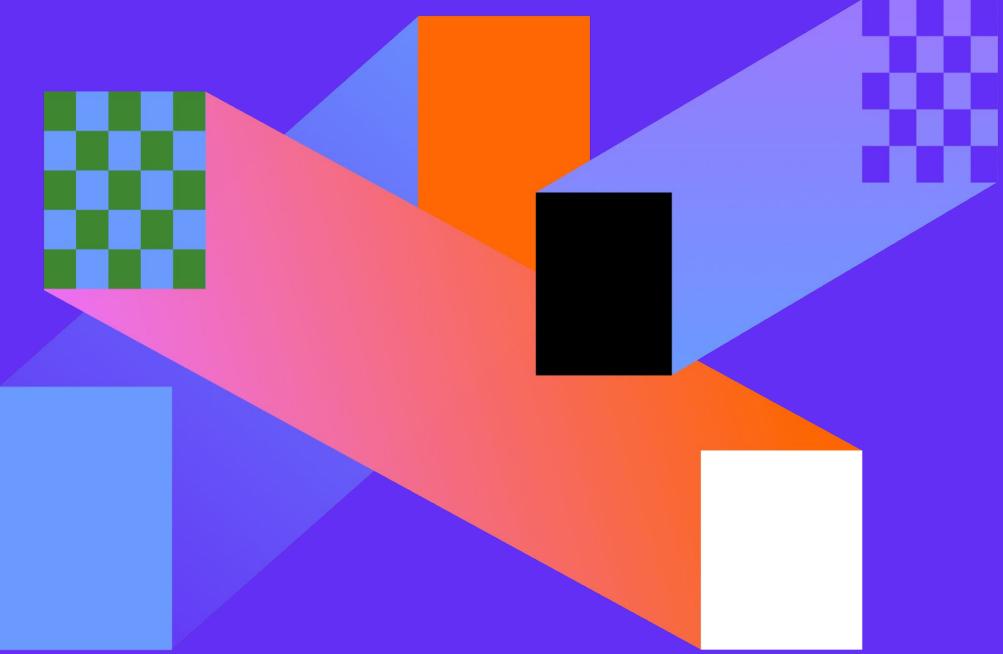
30 min Testing

15 min Documentation

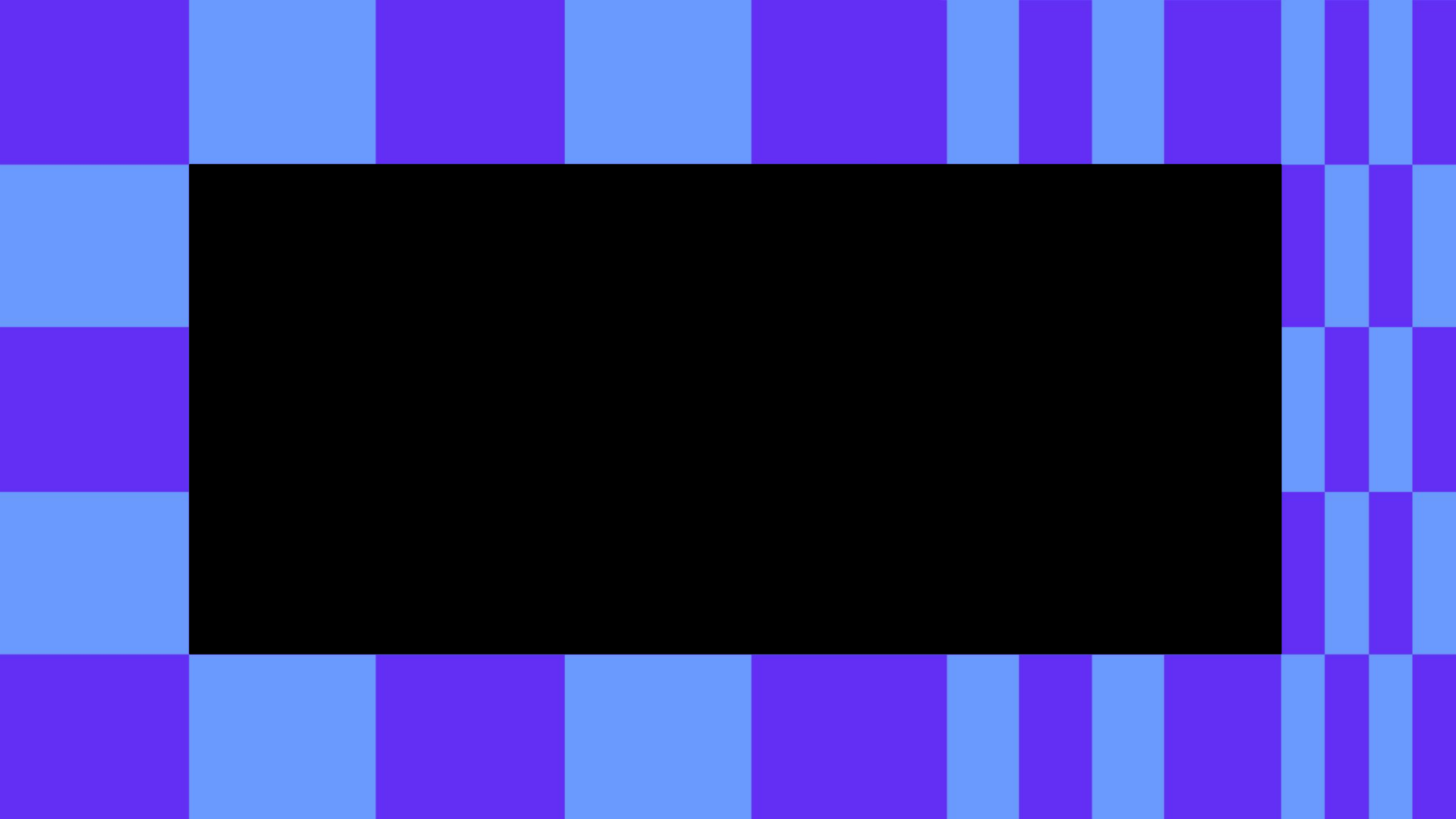
10 min *Break*

25 min Deployment and dbt Catalog

15 min Wrap Up



Data Testing



TESTING

What are tests?



Assertions

Assertions you make about your models to ensure data quality and integrity.



Validation

When your assertion is true, your test passes successfully.



Configuration

Configured easily in YAML files, right in your models directory.



Under the Hood

Behind the scenes, dbt tests are compiled into plain SELECT statements.

TWO TYPES OF TESTS

Singular vs. Generic Tests

Singular Tests

- Target one single model
- Not reusable
- Custom SQL stored in the **tests/** folder

Generic Tests

- Can target many models
- Code is reusable
- Defined on columns in YAML files
- Four built into dbt

EXAMPLE: NOT_NULL

A generic test in YAML

YAML SPECIFICATION

```
models:
  - name: orders
    columns:
      - name: order_id
        data_tests:
          - not_null
```

BEHIND THE SCENES (SQL)

```
select count(*)
from analytics.dim_customers
where customer_id is null
```

RUNNING TESTS

How do we run tests?

RUN ALL TESTS

dbt test

Executes every singular and generic test configured across your entire dbt project.

RUN SPECIFIC TESTS

dbt test --select <model>

Restricts test execution to a single designated model to save compute and time.

RUN SOURCE TESTS ONLY

dbt test --select source:*

Runs tests only on external data sources to validate raw data freshness and integrity.

RUN ALL IN DAG ORDER

dbt build

Runs models, tests, seeds, and snapshots in exact lineage order, halting on failures.

dbt BUILD

dbt build in one picture

THE UNIFIED COMMAND

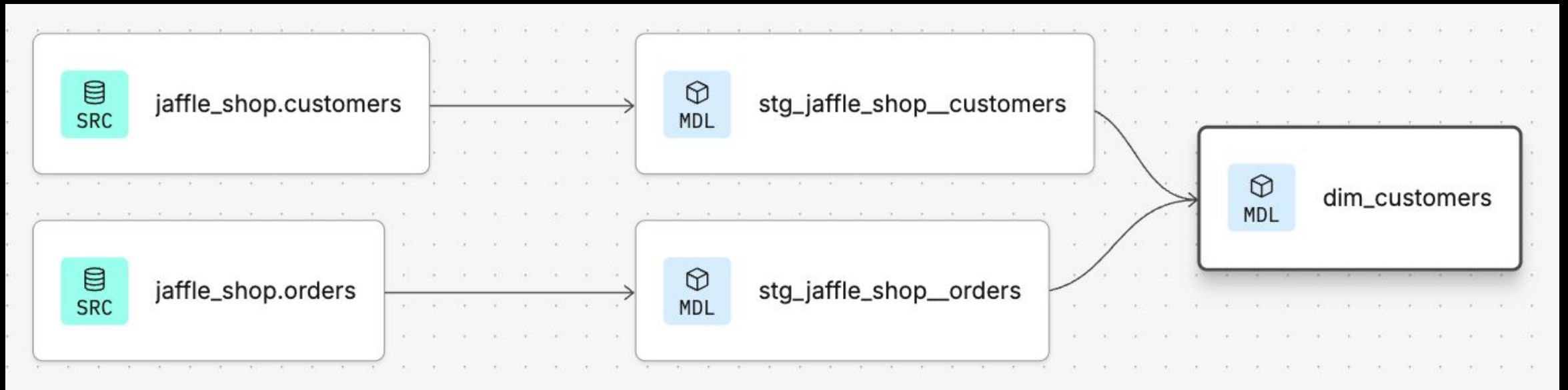
dbt build

Runs models, tests, seeds, and snapshots in exact DAG lineage order

LINEAGE PROTECTION

Halt on Failure

Halts execution immediately on failure to protect downstream nodes from running on bad data.



UNDER THE HOOD

What happens when you run `dbt test`?

01 // CONNECT

dbt connects to
your data
platform

02 // PARSE

dbt parses your
project

03 // CONSTRUCT

dbt iterates
through
resource files to
construct select
queries

04 // EXECUTE

dbt executes
the SQL for
each test

Zero rows =
pass

dbt'S BUILT-IN TESTS

unique

TEST ASSERTION

Every value in this column is unique.

COMPUTED SQL QUERY

```
select count(*) from (  
  select customer_id  
  from analytics.dim_customers  
  group by customer_id  
  having count(*) > 1  
)
```

dbt'S BUILT-IN TESTS

not_null

TEST ASSERTION

Every value in this column is not null.

COMPUTED SQL QUERY

```
select *  
from analytics.dim_customers  
where customer_id is null
```

dbt'S BUILT-IN TESTS

accepted_values

TEST ASSERTION

Every value in this column is from a given list.

YAML CONFIGURATION

```
models:
  - name: stg_jaffle_shop_orders
    columns:
      - name: status
        data_tests:
          - accepted_values:
              arguments:
                values:
                  ['placed', 'shipped',
                  'completed', 'returned',
                  'return_pending']
```

dbt'S BUILT-IN TESTS

relationships

TEST ASSERTION

Every value in this column exists in the column of another table.

YAML CONFIGURATION

```
models:
  - name: stg_jaffle_shop__orders
    columns:
      - name: customer_id
    data_tests:
      - relationships:
          arguments:
            to: ref('stg_jaffle_shop__customers')
            field: customer_id
```

demo_time

Live in dbt Platform

BEST PRACTICES

What should you test?

01 / Core Constraints

Test primary keys

Establish trust in your foundation. Every table's primary identifier should be verified.

`unique`

`not_null`

02 / Downstream Trust

Test field expectations

Verify column values and business expectations before they are used in downstream models.

- Check accepted values
- Prevent invalid business logic

TESTING SOURCES

You can (and should!) test sources

01 / CORE BENEFITS

Isolate failures early

Thoughtful source testing tells you exactly where pipeline errors originate.

- Verify if **source data assumptions** are no longer true
- Significantly **debug and resolve** issues faster

02 / SCHEMA DEFINITION

Configure source tests

```
sources:  
  - name: jaffle_shop  
    database: raw  
    schema: jaffle_shop  
    tables:  
      - name: customers  
        columns:  
          - name: id  
            data_tests:  
              - not_null  
              - unique
```

FEELING FRESH?

Define source freshness

01 / SLA MONITORING

Track data latency

Ensure your pipelines are updating as expected by configuring freshness checks.

- Define **warning & error thresholds** using count and period arguments
- Specify your update timestamp using the **loaded_at_field**

02 / FRESHNESS CONFIG

sources:

```
- name: jaffle_shop
  database: raw
  schema: jaffle_shop
  tables:
```

- ```
- name: customers
- name: orders
```

```
 config:
```

```
 freshness:
```

```
 warn_after: {count: 12, period: hour}
```

```
 error_after: {count: 24, period: hour}
```

```
 loaded_at_field: _etl_loaded_at
```

# then run:

```
dbt source freshness
```

## UNDER THE HOOD

# How does dbt check freshness?

### 01 / THE MECHANISM

## How dbt computes lag

dbt executes a query against your source table to determine the time elapsed since the last load.

- **Find last update:** Scans the `loaded_at` column to find the maximum timestamp.
- **Compare to current:** Safely compares it against the query execution time in UTC.

### 02 / EXECUTED SQL

```
select
max(_etl_loaded_at) as
max_loaded_at,
convert_timezone(
'UTC',
current_timestamp()
) as snapshotted_at
from raw.jaffle_shop.orders
```

# demo\_time

Live in dbt Platform

## DEBUGGING

# Why might a test fail?

### 01 / MODEL LOGIC

## SQL doesn't do what you intended

The query logic introduces unintended behavior or structural errors.

- Primary key duplicated
- Join introduces unexpected NULLs

### 02 / SOURCE DATA

## Assumptions are wrong

Source data structure contradicts original assumptions.

- Non-unique IDs
- 1:many instead of 1:1 relationship
- Missing data or unexpected NULLs

### 03 / DATA DRIFT

## Past truth no longer holds

Data evolved over time, breaking established invariants.

- Once-unique key is no longer unique
- New unhandled values (e.g. payment method)

## CADENCE

# When should you run tests?

### 01 / MANUAL

- When you first run a project
- During development

### 02 / AUTOMATED

- When you run dbt on a schedule
- When you want to merge code

## CHECK FOR UNDERSTANDING

```
models:
 - name: stg_stripe__payment
 columns:
 - name: payment_id
 data_tests:
 - not_null
 - unique
```

**Q:** What's being tested, and what does failure tell us?

**A:**

**not\_null + unique** on  
payment\_id

Failure means there are **duplicates** or a  
**null value** in the primary key column.

## CHECK FOR UNDERSTANDING

```
models:
 - name: stg_stripe__payment
 columns:
 - name: payment_method
 data_tests:
 - accepted_values:
 arguments:
 values:
[credit_card, paypal, ach_transfer]
```

**What tests are running, on what, and what does failure tell us?**

accepted\_values on  
payment\_method - failure  
means an unexpected value  
appeared.

# Try it yourself

## > Hands-on

- 01 Create `_models.yml` in the staging folder
- 02 Define `unique` / `not_null` on all primary keys (models & sources)
- 03 Add `relationships` on `customer_id`, `accepted_values` on `status`
- 04 Run tests and source freshness

## ✓ You should be able to

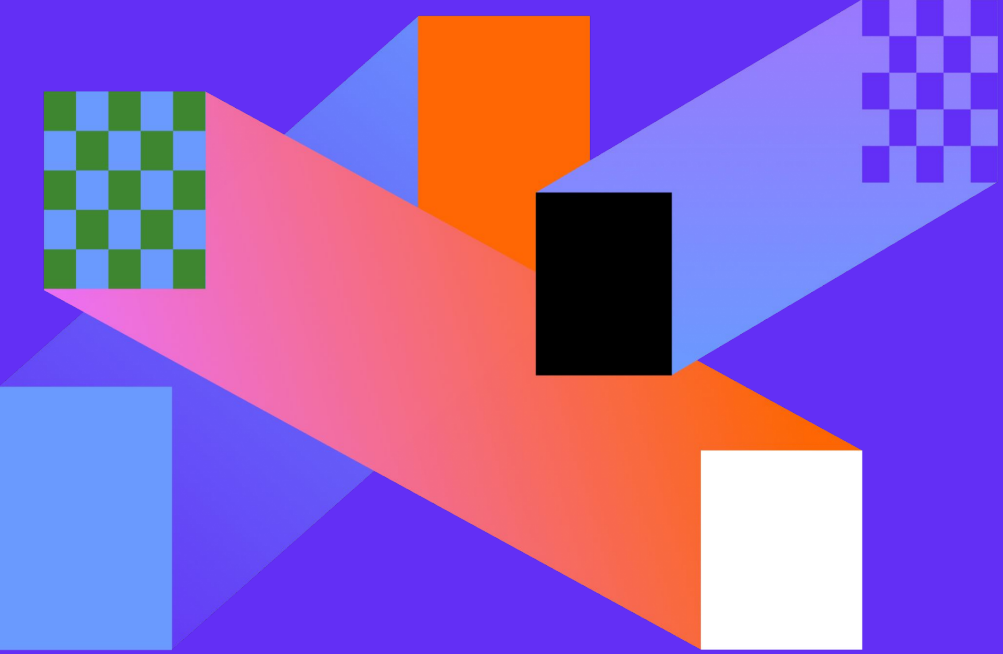
- ◆ Add tests to models & sources
- ◆ Run tests (all / one / just sources)
- ◆ Explain how dbt tests work
- ◆ Find compiled SQL for tests

# Questions?

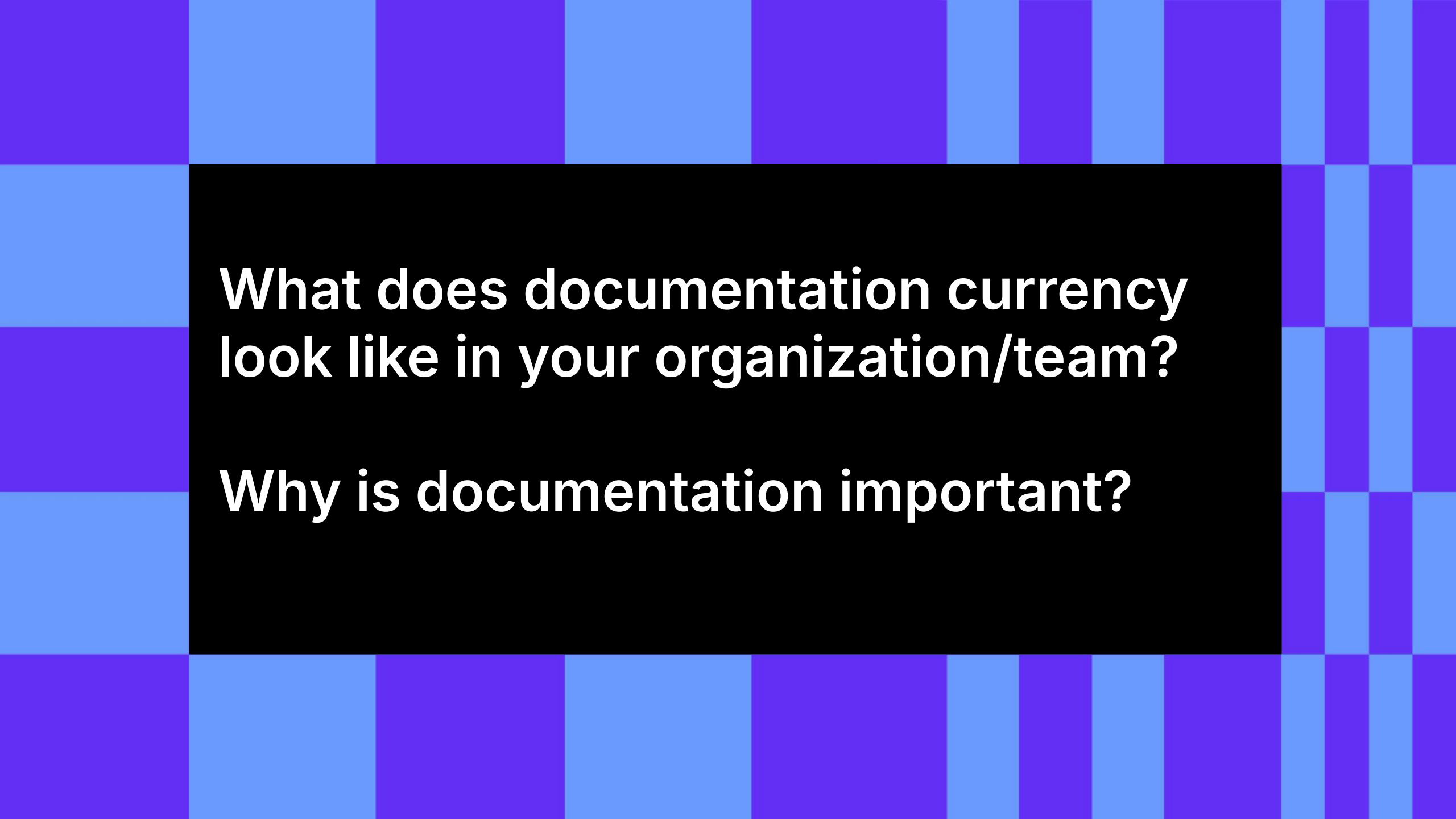
## TODAY'S AGENDA

# Agenda

- 30 min Introduction to dbt
- 15 min Setting up dbt
- 15 min *Break*
- 20 min Sources
- 30 min Data Modeling in dbt
- 15 min *Break*
- 30 min Testing
- 15 min Documentation**
- 10 min *Break*
- 25 min Deployment and dbt Catalog
- 15 min Wrap Up



# Documentation



**What does documentation currency  
look like in your organization/team?**

**Why is documentation important?**

## WHY IT MATTERS

# Documentation in dbt

### 01 / YAML-BASED

## Docs Live with Code

Stored directly alongside your models in YAML files—making documentation seamless to find, review, and keep current.

### 02 / KNOWLEDGE RETENTION

## Preserve Decision Context

Help team members remember the critical business logic and technical decisions behind complex joins and aggregations.

### 03 / ONBOARDING

## Self-Service Discovery

Enable newcomers to independently answer questions about table schemas, column meanings, and data origins.

### 04 / GOVERNANCE

## Explicit Data Quality

Raise trust and dataset standards by making technical intent, assumptions, and edge-case definitions explicit.

## ADDING DESCRIPTIONS

# Descriptions on models, sources & columns

### 01 / MODEL LEVEL

#### Dataset Purpose

Document what a single row represents in the model (e.g., 'One record per order').

### 02 / COLUMN LEVEL

#### Field Semantics

Clarify business definitions, units (USD), and valid status values directly in code.

### 03 / DATA TESTS

#### Inline Validation

Pair documentation with automated tests like unique, not\_null, and accepted\_values.

```
models/schema.yml

models:
 - name: stg_jaffle_shop_orders
 description: "One record per order"
 columns:
 - name: order_id
 description: "Distinct primary key"
 data_tests: [unique, not_null]
 - name: order_status
 description: "placed, shipped, completed, ..."
 data_tests:
 - accepted_values:
 values: ['placed', 'shipped', 'completed',
 'return_pending', 'returned']
 - name: amount
 description: "Amount in USD"
```

## DOC BLOCKS

# Long-form descriptions in Markdown

### 01 / LOCATION

#### Stored in .md Files

Added directly to .md files inside your models/ directory.

### 02 / FLEXIBILITY

#### Multiple Doc Blocks

A single file can house multiple doc blocks to organize related descriptions.

### 03 / FORMATTING

#### Rich & Reusable

Great for longer explanations, Markdown tables, and reused descriptions.



models/docs.md

```
{% docs order_status %}
```

One of the following values:

```
| status | definition |
```

```
|-----|-----|
```

```
| placed | Order placed, not yet shipped |
```

```
| shipped | Shipped, not yet delivered |
```

```
| completed | Received by customer |
```

```
| return_pending | Return requested |
```

```
| returned | Item returned |
```

```
{% enddocs %}
```

## REFERENCING DOC BLOCKS

# Use doc() instead of pasting text

### 01 / SYNTAX

#### doc() Function

Reference doc blocks inside schema YAML files using Jinja syntax.

### 02 / DRY PRINCIPLE

#### Single Source

Avoid repeating long descriptions across multiple models.

### 03 / MAINTAINABILITY

#### Instant Updates

Updating the markdown doc block updates all references automatically.

```
models/schema.yml

models:
 - name: stg_jaffle_shop__orders
 columns:
 - name: status
 description: "{{ doc('order_status') }}"
 data_tests:
 - accepted_values:
 values: ['placed',
 'shipped', 'completed',
 'return_pending', 'returned']
```

## GENERATING DOCUMENTATION

# What dbt takes into account

### 01 / PROJECT KNOWLEDGE

## Internal Project Context

Everything dbt knows natively about your codebase:

- **Model Descriptions:** Written in YML or doc blocks
- **Model Dependencies:** Lineage graph & refs
- **Model SQL:** Transformation code
- **Sources & Tests:** Data freshness & assertion rules

### 02 / PLATFORM METADATA

## Data Platform Info

Additional details introspected live from your data warehouse:

- **Column Names & Types:** Inferred system data types
- **Table Statistics:** Row counts, size, and storage stats
- **Undocumented Columns:** Captured automatically even if omitted in YML

## BEST PRACTICES

# What should I document?

### 01 / INTENT & PURPOSE

**What is the data used for?**

### 02 / OWNERSHIP & EXPERTISE

**Who knows the most about this data - technically and business-wise?**

### 03 / LOGIC & RATIONALE

**Why were these transformations made?**

### 04 / CAVEATS & LIMITATIONS

**Any causes of confusion, limitations, or caveats**

# demo\_time

Live in dbt Platform

## KNOWLEDGE CHECK

# Try it yourself

### Hands-on

- Describe all staging models & their primary key columns
- Create a useful doc block
- Link the doc block to a column's description

### You should be able to

- Add descriptions to your dbt project

## KNOWLEDGE CHECK

# Try it yourself

### HANDS-ON

- Describe all staging models & their primary key columns
- Create a useful doc block
- Link the doc block to a column's description

### YOU SHOULD BE ABLE TO

- **Add descriptions to your dbt project**

# Questions?

# break\_time

10 minutes

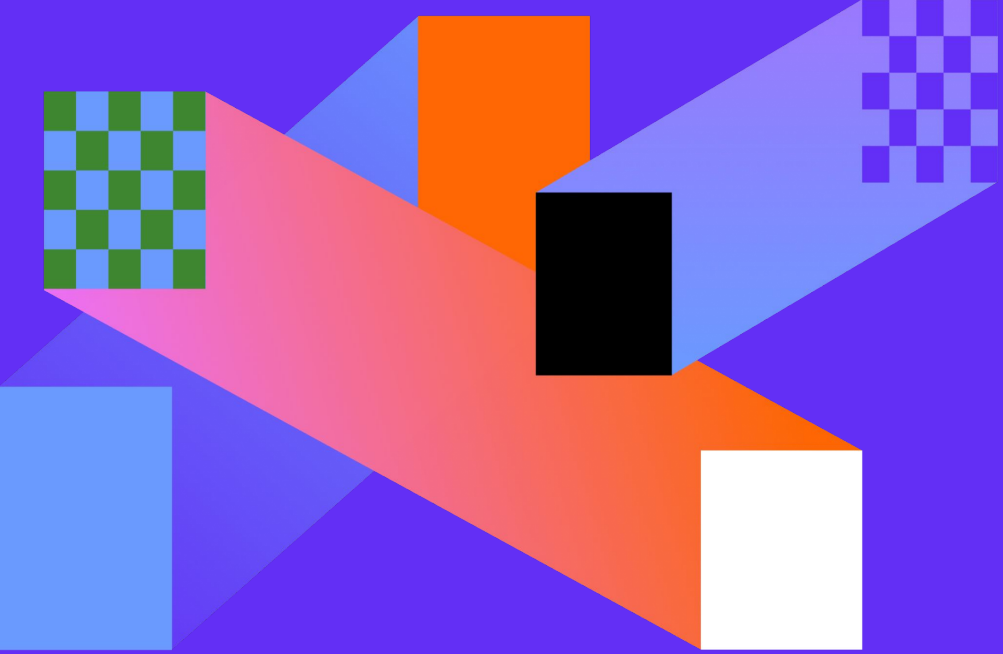
## TODAY'S AGENDA

# Agenda

- 30 min Introduction to dbt
- 15 min Setting up dbt
- 15 min *Break*
- 20 min Sources
- 30 min Data Modeling in dbt
- 15 min *Break*
- 30 min Testing
- 15 min Documentation
- 10 min *Break*

## 25 min **Deployment and dbt Catalog**

- 15 min Wrap Up



# Deployment

# We've built our first models... now what?



## Teammate Access

How will teammates access them? They're currently locked in your personal dev schema.



## Data and Model Changes

What happens when the underlying data or models get updated?

# Deploying jobs with the dbt platform

## Reliability & Automation

- **Latest Project Version:** Always pulls the latest version of your project.
- **Easy CI/CD:** Easy to build on a schedule, PR, or merge.
- **Documentation:** Easy to generate and share project documentation.

## Security & Operations

- **Secure Access:** Robust and secure access control structure.
- **Rich Notifications:** Email/Slack notifications with detailed logs on failure.
- **Run Artifacts:** Persistent run artifacts for reliable later comparison.

# Deployment with the dbt platform

## Deployment Environment

- **Target Environment:** A dedicated space configured to run your production models.
- **Credentials:** Secure connections to execute commands against your warehouse.

## The Job & Trigger

- **Job Commands:** Specify exact dbt commands to run (e.g., dbt run, dbt test).
- **Triggers:** Set up automated schedules or events to trigger jobs.

# Considerations for running jobs

## Who's in charge?

- Secure access structure (RBAC)
- Alerts when something goes wrong (email/Slack)
- A debugging & fix process, with a point person

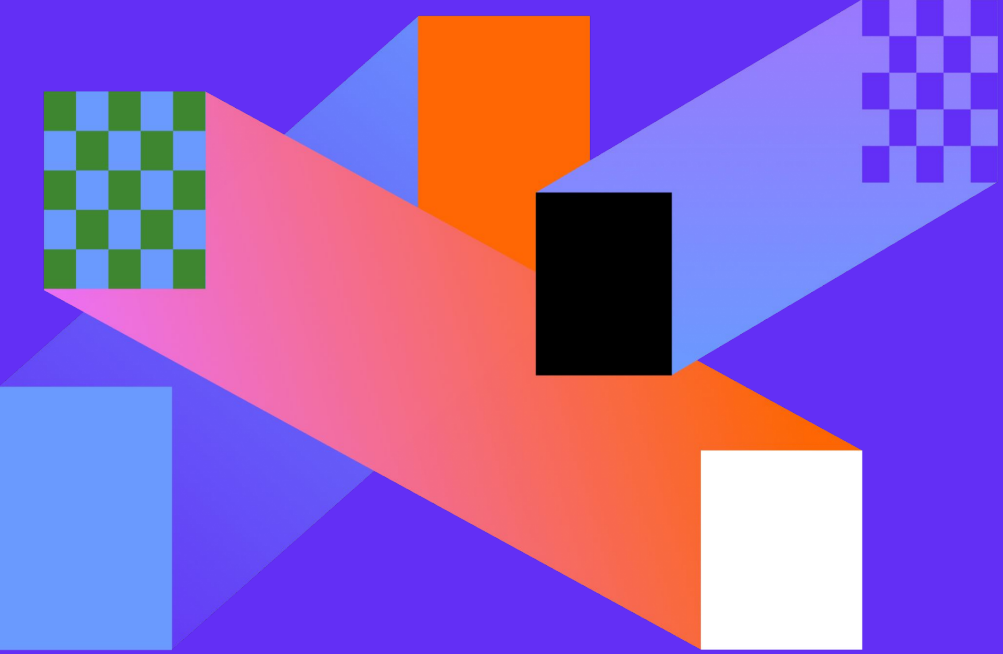
## Type of job

- **CI:** build & test modified code on pull request
- **CD:** builds, tests, and deploys code on merge
- **Deploy:** scheduled or triggered regular runs

# demo\_time

Live in dbt Platform

# Questions?



# dbt Catalog

## PREREQUISITES

# What you need for Catalog



### Plan Eligibility

Available on Starter, Enterprise, and Enterprise+ plans.



### Production Env

A production deployment environment must be created and configured.



### Successful Run

Requires at least one successful job run completed in that environment.

# demo\_time

Live in dbt Platform

# Try it yourself

## Hands-on

- Create a production deployment environment on your training branch
- Kick off a `dbt build` job with `dbt docs generate` and `source freshness`
- Explore: models, lineage, trends, and recommendations in dbt Catalog

## You should be able to

- Explain why you deploy your dbt project
- List deployment options
- Deploy with the dbt platform

## CHECK FOR UNDERSTANDING

### Question

**A developer clicks Catalog but sees no resources.**

*Why might this be?*

### Answer

**No successful prod/staging job with `docs generate` ran yet.**

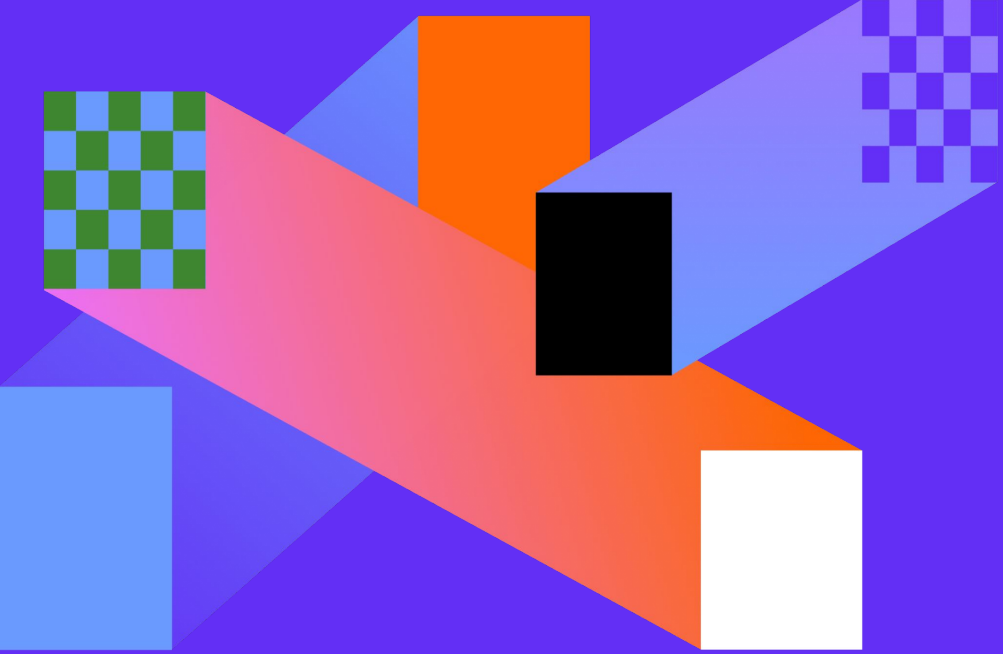
Catalog uses the Discovery API from job runs, and `docs generate` in the job ensures full metadata populates in Catalog.

# Questions?

## TODAY'S AGENDA

# Agenda - We did it! 🎉

|        |                            |
|--------|----------------------------|
| 30 min | Introduction to dbt        |
| 15 min | Setting up dbt             |
| 15 min | <i>Break</i>               |
| 20 min | Sources                    |
| 30 min | Data Modeling in dbt       |
| 15 min | <i>Break</i>               |
| 30 min | Testing                    |
| 15 min | Documentation              |
| 10 min | <i>Break</i>               |
| 25 min | Deployment and dbt Catalog |



# Getting Started with dbt: Wrap Up

## HOUSEKEEPING

# After today

01

### Practice

Continue to practice and build on what you have learned today.

02

### Platform Account

Log in with your platform account, or register a new one **(don't re-register if you already have one!)**.

03

### Sandbox Expiry

Your temporary dbt Platform sandbox account expires in about a week.

# KEEP LEARNING

## Dive Deeper!

01

### Learn On-Demand

Explore more content at [learn.getdbt.com](https://learn.getdbt.com) to continue your learning journey.



02

### Get Certified

Earn dbt Developer and/or Architect certification.



03

### Community Slack

Join the dbt Community Slack to connect with data peers and get help.



# Thank you!

Please take 5 minutes for the post-session survey - it helps us improve.

<https://tinyurl.com/dbt-summit-survey>

The dbt logo, an orange stylized 'X' shape with a small black diamond in the center.

# dbt SUMMIT